OnlineSoccer: Compact Video Captioning with a Causal Spatiotemporal Projector

Xin Huang*, Tianle Xu†, Jiawen Huang*, Shijie Jia*
*Dalian Jiaotong University, Dalian, China
†Fudan University, Shanghai, China
20252340@djtu.edu.cn, xutianle@fudan.edu.cn,
wodenaozi@gmail.com, jsj@djtu.edu.cn

***Abstract*—Captioning soccer clips can provide reusable drafts for collaborative sports-media production. Dense visual prefixes, however, make each draft expensive in language-model computation and memory. We present OnlineSoccer, a complete-clip captioning model whose visual interface aggregates temporal evidence before selecting outputs. Temporal Mamba Proj combines reduced-width causal processing, spatial pooling, and channel packing; an inference-time stride adjusts the visual-prefix budget without retraining. On centered clips from MatchTime-corrected SoccerNet Caption, the unsampled interface uses 112 visual tokens instead of 1,792 and reaches 37.36 CIDEr, a 7.91% relative improvement over the dense baseline. The test-informed stride-two configuration uses 56 tokens and reaches 37.54 CIDEr, exceeding pooling at the same inference budget by 19.55%. In a separate temporal-only sweep, mean CIDEr remains similar across an eightfold prefix reduction: the score range is 0.81% of the stride-one score, compared with a 14.36% decrease for direct sampling. These results support aggregation before output selection as a way to combine compact visual prefixes with adjustable inference budgets.**

***Index Terms*—video-language models, token compression, state-space models, causal sequence modeling, soccer captioning**

I. Introduction

Soccer-caption generation can supply drafts that commentators and editors review against source footage and reuse in media production. SoccerNet-Caption and MatchTime provide a basis for studying this generation component [1], [2]. Its computational cost matters when several clips share a language-model service: each visual prefix consumes attention computation and key–value (KV) storage. Compressing that prefix must preserve the temporal context needed to describe actions, including the build-up to a pass, shot, or clearance.

A restricted token budget creates two related choices: how to represent each frame and which temporal outputs to retain. Direct sampling shortens the sequence but omits unsampled features from the language-model input. Aggregating evidence before output selection instead lets retained representations depend on preceding observations. Mamba-based projectors, including STORM [3], demonstrate the relevance of temporal processing to video token compression. For soccer captioning, we examine how this ordering works with spatial pooling and channel packing to reduce both the projector's working width and its output-token count.

OnlineSoccer uses Temporal Mamba Proj as a visual interface to Qwen2.5-VL [4]. Every time unit updates a unidirectional state-space model, while an inference-time stride controls which final-block outputs reach the language model. This separates temporal processing from the decoder's visual-token budget: retained outputs can carry information from preceding units whose own outputs are omitted. Spatial pooling and channel packing further shorten the prefix. We evaluate captions generated after complete clips have been processed (Fig. 1).

Our contributions are as follows:

- We design a Qwen2.5-VL visual interface that combines 896-dimensional temporal processing, spatial pooling, and within-unit channel packing to produce compact 3,584-dimensional tokens.
- We demonstrate a quality–compression trade-off in which the unsampled interface improves CIDEr by 7.91% over the dense baseline at 16× compression. The test-informed 32× configuration improves CIDEr by 19.55% over pooling at the same inference budget.
- We evaluate budget adjustment without retraining through temporal-only and full-interface sampling comparisons. Across strides 1–8, the temporal-only score range is 0.81% of its stride-one value, while direct sampling decreases by 14.36%.

II. Related Work

A. Visual Projectors and Video Token Compression

Early vision-language systems used learned interfaces to align visual and language backbones. Flamingo introduced gated cross-attention for interleaved visual and language inputs, while BLIP-2 used a query-former bottleneck to connect a frozen image encoder to a large language model [5], [6]. LLaVA showed that a simple learned projector suffices for strong instruction following [7], and Honeybee emphasized locality-preserving visual projectors [8]. For video, TokenLearner learns a small set of adaptive spatial tokens per frame, while TESTA aggregates similar frames and similar patches within a frame [9], [10]. LLaMA-VID represents each frame with as few as two tokens for long-video understanding [11], and FastV prunes visual tokens inside the language model at inference [12]. MovieChat and VideoChat-Flash maintain compact representations or hierarchical memories for long videos [13], [14]. Closest to our adjustable budget, Matryoshka

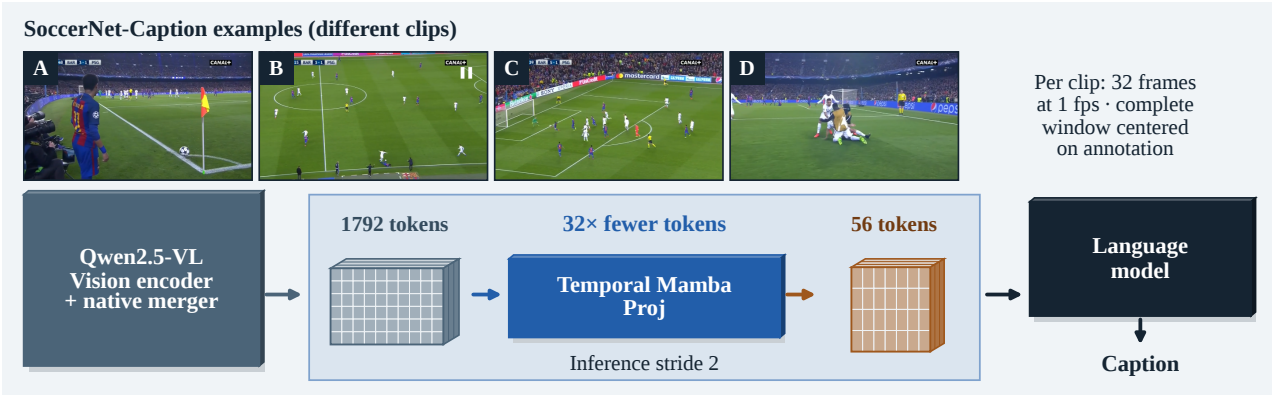


Fig. 1. Complete-clip soccer captioning with a compact visual prefix. For each 32-frame window centered on its annotation timestamp, Temporal Mamba Proj compresses 1,792 native visual tokens to 56 at inference stride two. The four photographs are examples from different clips in SoccerNet-Caption [1], Fig. 7(a–d); the processing flow applies to each clip separately.

multimodal models encode visual tokens at nested granularities so that one model serves several token counts at inference [15]; we instead obtain multiple budgets from a single causal projector by changing only the output stride.

LongVU performs spatiotemporal adaptive compression for long-video understanding [16]. STORM places Mamba layers between the visual encoder and language model, combines temporal mixing with spatial pooling, and subsamples outputs at inference [3]. Alongside its bidirectional projector, it evaluates a unidirectional variant with recurrent state reuse. OnlineSoccer shares temporal aggregation before output selection, with a specific interface organization: width reduction and spatial pooling precede the temporal blocks, and channel packing restores the language-model width afterward. Our evaluation focuses on this integrated interface and its sampling behavior in soccer captioning, using dense and pooling baselines.

B. Causal Sequence Modeling and Streaming Video

Mamba introduced selective state-space models with input-dependent state updates, Mamba-2 connected the family to structured state-space duality, and Mamba-3 develops these principles further [17], [18], [19]. In vision, Vision Mamba and VMamba adopt state-space backbones for image representation [20], [21], and VideoMamba extends state-space modeling to visual temporal representations [22]. We place a causal state-space block inside the projector, where it aggregates visual evidence before token selection and language decoding.

VideoLLM-online and StreamingTOM study online video understanding and streaming token compression [23], [24]. HERMES organizes KV cache as hierarchical memory [25]; StreamingLLM, H2O, and SnapKV control language-model cache growth [26], [27], [28]. These cache-side methods complement our focus on the visual prefix itself: Temporal Mamba Proj controls which aggregated features enter the language model.

C. Soccer Captioning

SoccerNet established large-scale soccer video understanding through action spotting [29], and SoccerNet-Caption introduced dense natural-language commentary generation for soccer broadcasts [1]. MatchTime addressed temporal misalignment through corrected commentary timestamps [2]. UniSoccer and TimeSoccer broaden soccer video understanding and multimodal commentary generation [30], [31]. We use MatchTime-corrected SoccerNet Caption to compare visual interfaces under a shared data, fine-tuning, and decoding protocol.

III. Causal Spatiotemporal Projector

A. Problem Definition

Let a clip contain T_f sampled frames. The Qwen2.5-VL vision transformer (ViT) groups every two consecutive frames along the temporal axis, so it emits $T = T_f/2$ *time units*, each carrying N spatial tokens that already reflect the encoder's own 2×2 spatial patch merging. Writing the encoder output as

$$X^{(0)} \in \mathbb{R}^{T \times N \times d_v}, \qquad d_v = 3584, \tag{1}$$

T is the number of time units and N the number of spatial tokens per unit. With 32 frames at a processed resolution of 224×392 pixels (height $\times$ width), 14×14 patches and native spatial merging yield $T = 16$ and an 8×14 grid, or $N = 112$ tokens per unit. The projector F produces a shorter sequence at the language-model input width. Let τ_j be the latest input time-unit index associated with output token Y_j. Projector causality requires

$$Y_j = F_j(X^{(0)}_{1:\tau_j}), \qquad \frac{\partial Y_j}{\partial X^{(0)}_t} = 0 \quad \forall t > \tau_j. \tag{2}$$

An output token is available after all its constituent time units have arrived. Causality is defined at the projector input; a frame-level streaming implementation must also respect this temporal boundary in the upstream feature extractor.

For a reference caption $w = (w_1, \ldots, w_M)$, the training objective is the standard autoregressive negative log-likelihood

$$\mathcal{L}_{\text{cap}} = -\sum_{i=1}^{M} \log p(w_i \mid w_{<i}, F(X^{(0)})). \tag{3}$$

B. Architecture

The input first passes through a learned linear projection from $d_v = 3{,}584$ to $d_v/4 = 896$:

$$X^{(1)} = X^{(0)} W_{\text{in}} + b_{\text{in}}, \qquad W_{\text{in}} \in \mathbb{R}^{3584\times 896}. \tag{4}$$

The projection preserves token count. A 2×2 mean pool with stride two then reduces each unit's 8×14 grid to 4×7. Figure 2 summarizes the data path.

Each pooled grid is flattened in row-major order, and the units are concatenated chronologically into a 448-token sequence. Four causal Mamba-3 blocks process this sequence, each with a SwiGLU residual branch. All blocks use working width 896, state dimension 1,024, expansion factor 2, and head dimension 64. Abstractly, the state-space path follows

$$h_s = A_s h_{s-1} + B_s u_s, \tag{5}$$

$$m_s = C_s h_s + D_s u_s, \tag{6}$$

where s indexes flattened tokens and the operators denote an abstract causal state update, rather than the full Mamba-3 parameterization. The block combines the state-space module M with residual connections and a SwiGLU branch:

$$e_s = u_s + M(\mathrm{LN}(u_s)), \tag{7}$$

$$\mathrm{SwiGLU}(z) = \mathrm{SiLU}(zW_g) \odot (zW_v), \tag{8}$$

$$v_s = e_s + \mathrm{SwiGLU}(\mathrm{LN}(e_s))W_o. \tag{9}$$

The SwiGLU hidden width is 1,792, twice the 896-dimensional input width.

Training uses all time-unit outputs. At inference, the four blocks process the complete sequence before final-block outputs are selected. Stride r retains units $1, 1+r, 1+2r, \ldots \le T$; stride two selects units $1, 3, \ldots, 15$. Every input updates the state, and selected readouts carry the accumulated preceding context into the language model. Units after the last retained unit cannot influence earlier readouts. Changing r requires no retraining.

Within each retained unit, every four consecutive 896-dimensional block outputs in row-major order are concatenated to recover the 3,584-dimensional language-model width. Groups may cross row boundaries, but never time-unit boundaries. Each unit's 28 outputs become seven tokens without padding or truncation. The output length is

$$K(r) = 7\,\lceil 16/r \rceil\,. \tag{10}$$

For the tested strides dividing 16, compression relative to 1,792 dense tokens is $16r$. Training uses $K(1) = 112$ tokens; inference at stride two uses $K(2) = 56$. For the language model's multimodal rotary position embedding (MRoPE), each packed unit uses a new 1×7 spatial grid and retains its original temporal coordinate, leaving temporal indices sparse after sampling.

The temporal scan processes every input token at each stride. Increasing r therefore changes decoder input length while preserving the number of temporal-state updates. The sampling comparisons measure how well this aggregation preserves caption-relevant information at smaller output budgets.

IV. Experiments

A. Dataset and Training Protocol

We use the official SoccerNet-Caption train, validation, and test splits [1] with MatchTime-corrected timestamps [2]. Splits are match-disjoint. All quality experiments sample 32 frames at 1 frame/s from a window centered on each corrected timestamp, covering approximately 16 seconds before and after it. The Qwen processor resizes the original 224×398 frames to 224×392 without a separate manual resize. Captions are generated after the complete clip is processed, including observations after the annotation time.

All models freeze the vision transformer (ViT) and fully fine-tune the language model. Dense and mean-pooling baselines train the native patch merger; the full causal and bidirectional projectors freeze it and train the added interface. Each pooling ratio is trained separately and held fixed between training and evaluation. Our full projector trains with 112 visual tokens and uses 56 at stride-two inference, whereas $32\times$ pooling uses 56 in both phases. The comparison matches inference budgets. All independent runs use AdamW at 10^{-5}, cosine annealing, 20 optimizer-step warmup, global batch 16, and 50 epochs on eight A800-80GB GPUs, with two samples per GPU.

Checkpoints are selected by validation CIDEr. The output stride was chosen after inspecting test results from the temporal-only and full-model sweeps; stride-two results are therefore conditional on test-informed selection. Other settings were fixed beforehand or selected on validation data. Full-model scores beyond the reported strides one and two are unavailable.

All configurations use the prompt "Describe the soccer action in this video clip." and the same Qwen2.5-VL-7B-Instruct default generation settings, without explicit decoding or length overrides. BLEU-4 (B-4), METEOR, ROUGE-L (R-L), and CIDEr [32] are computed with the SoccerNet-Caption official evaluation script. Every quality score is a mean over the same five seeds; displayed ± values are standard errors (SE). The retained results consist of aggregate means and reported SEs; per-seed values needed to reconstruct the uncertainty estimates are unavailable.

B. Main Results

Table I shows that Temporal Mamba Proj improves all four caption metrics over the dense baseline while reducing the visual prefix 32-fold. CIDEr reaches 37.54 ± 0.32 versus 34.62 ± 0.28, an 8.43% relative improvement. This operating point combines temporal aggregation with stride-two output sampling.

a) Comparison with pooling: At the same 56-token inference budget, Temporal Mamba Proj obtains 37.54 CIDEr versus 31.40 for mean pooling, a 19.55% relative improvement. Across pooling configurations, CIDEr changes from 34.08 at $4\times$ compression to 31.40 at $32\times$. The learned interface supports a shorter prefix with higher caption quality. The following comparisons examine temporal aggregation and the contributions of the compression stages.

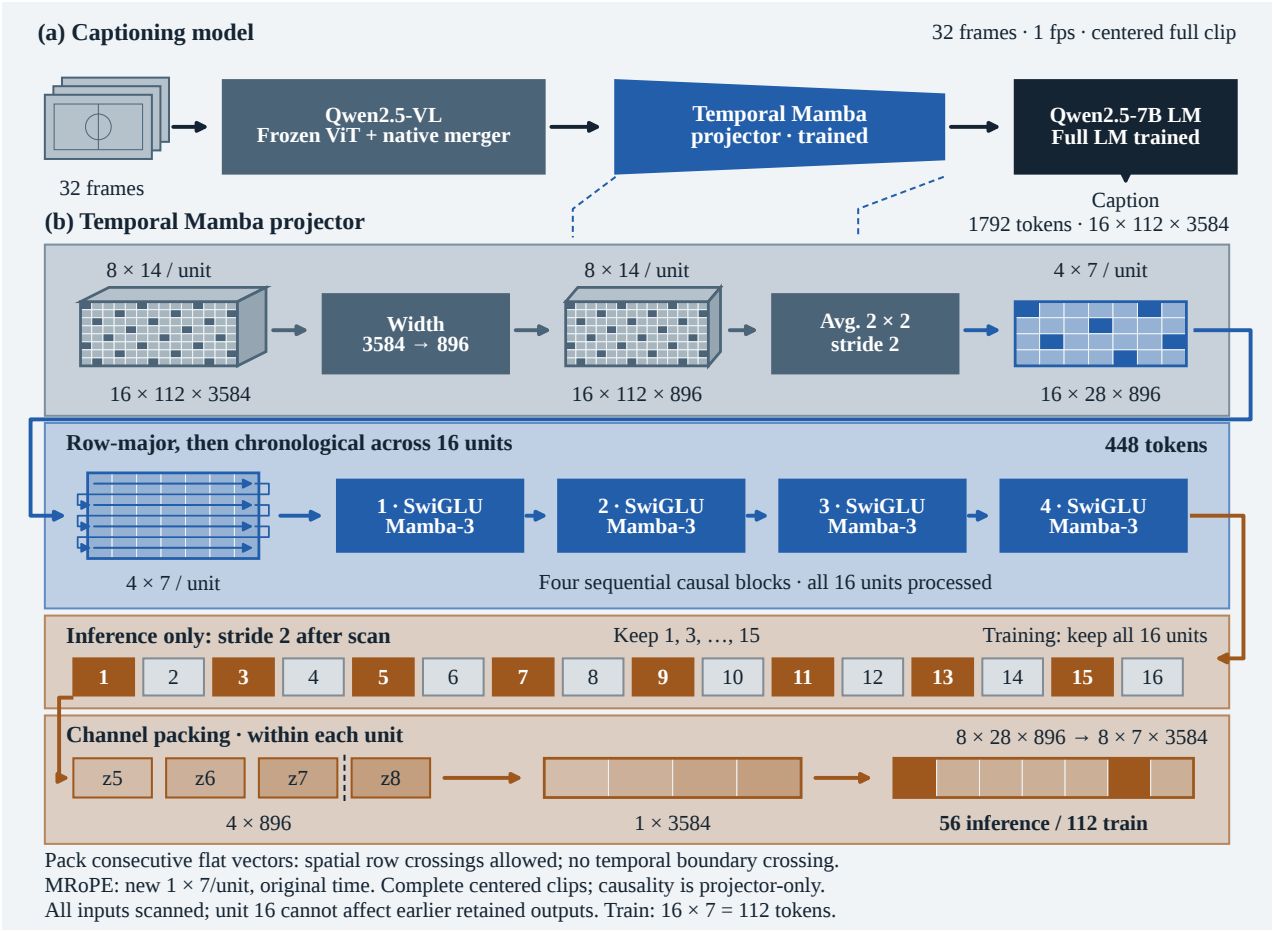


Fig. 2. The 32-frame architecture. The frozen ViT and native merger produce 16 temporal units. Width projection and spatial pooling yield 448 tokens; four SwiGLU–Mamba-3 blocks scan their flattened sequence chronologically, with row-major spatial order within each unit. Inference retains units $1, 3, \ldots, 15$, then concatenates each group of four consecutive row-major tokens within a unit along channels, producing 56 language-model tokens. Training retains all 16 units and produces 112 tokens. The projector and language model are trained; tensor shapes below selection show inference.

TABLE I

MAIN RESULTS ON MATCHTIME-CORRECTED SOCCERNET CAPTION. RED. DENOTES INFERENCE-TIME TOKEN COMPRESSION. EVERY METRIC IS A FIVE-SEED MEAN; CIDER INCLUDES STANDARD ERROR (SE). THE PROJECTOR'S STRIDE WAS SELECTED USING TEST RESULTS. THE 32× METHODS MATCH INFERENCE BUDGETS.

Method	Red.	B-4	METEOR	R-L	CIDEr
Qwen2.5-VL-7B (dense)	1×	6.82	21.28	26.36	34.62 ± 0.28
Mean pooling					
Spatial 4×	4×	6.61	21.07	25.98	34.08
+ temporal 2×	8×	6.41	20.67	25.93	33.78
+ temporal 4×	16×	6.18	20.22	25.05	33.14
+ temporal 8×	32×	5.84	19.43	24.62	31.40
Temporal Mamba Proj	32×	**8.14**	**25.93**	**28.01**	**37.54 ± 0.32**

TABLE II

TEMPORAL-ONLY SAMPLING SWEEP WITHOUT SPATIAL POOLING OR CHANNEL PACKING. CIDER VALUES ARE FIVE-SEED MEANS. WITHIN EACH SEED, EACH VARIANT REUSES ITS OWN CHECKPOINT ACROSS STRIDES.

Stride	Vis. tokens	No temporal block	Causal Mamba
1	1,792	34.62	37.02
2	896	34.58	37.31
4	448	33.52	37.32
8	224	29.65	37.16

C. Inference-Time Budget Adjustment

The temporal-only comparison omits spatial pooling and channel packing. Its added module projects 3,584-dimensional features to width 896, applies four SwiGLU–Mamba-3 blocks with the full model's internal settings, and projects back to 3,584. Unlike the full model, this variant trains the native patch merger together with the temporal module and language model. Direct sampling reuses the dense baseline checkpoint. Both variants train without temporal sampling and, within each seed, reuse their respective checkpoints at $r \in \{1, 2, 4, 8\}$. At each stride their output budgets match. This tests the added temporal module as a whole, including its learned projections.

Table II and Fig. 3 show that increasing the stride from 1 to 8 reduces the visual prefix from 1,792 to 224 tokens. Direct sampling decreases CIDEr by 14.36%, from 34.62 to 29.65. With temporal aggregation, scores span 37.02–37.32, a range equal to 0.81% of the stride-one score. At stride eight, the temporal module reaches 37.16 CIDEr, a 25.33% relative advantage over direct sampling at the same token budget.

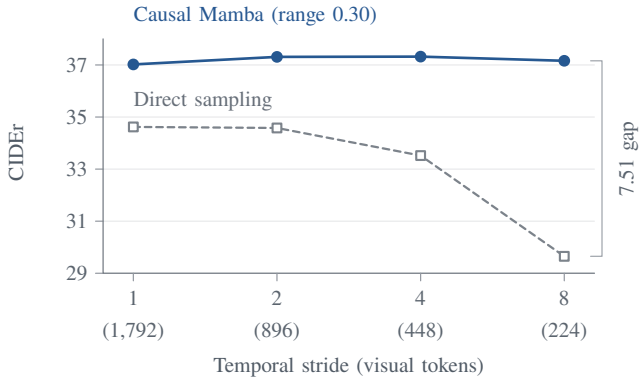


Fig. 3. Temporal-only sampling, without spatial pooling or channel packing. The complete temporal module, including its input/output projections, is compared with direct sampling at matched output budgets. All points are five-seed mean CIDEr scores on the absolute scale. The temporal module varies by 0.30 across strides 1–8, while direct sampling loses 4.97; the gap at stride 8 is 7.51.

TABLE III
PROGRESSIVE ABLATION. RED. DENOTES INFERENCE-TIME TOKEN COMPRESSION. ROWS 2–5 CONSTRUCT THE COMPRESSED INTERFACE; THE BASELINE USES THE NATIVE PATCH MERGER. THE LAST ROW APPLIES SAMPLING TO ROW 3 WITHOUT A TEMPORAL BLOCK. VALUES ARE FIVE-SEED MEANS; CIDER SE IS SHOWN WHERE AVAILABLE.

Configuration	Red.	B-4	METEOR	R-L	CIDEr
Baseline	1×	6.82	21.28	26.36	34.62 ± 0.28
+ Width proj. + packing	4×	6.91	24.36	25.43	35.21 ± 0.37
+ Spatial pool	16×	7.18	25.12	25.47	35.94 ± 0.21
+ SwiGLU–Mamba-3	16×	7.64	**25.98**	27.64	37.36 ± 0.30
+ Temporal sample	32×	**8.14**	25.93	**28.01**	**37.54 ± 0.32**
Row 3 + temporal sample	32×	6.99	24.79	24.96	35.44

D. Progressive Ablation

Table III traces the full interface's quality–compression trade-off. Width projection and channel packing reach 4× compression with 35.21 CIDEr. Spatial pooling increases compression to 16× with 35.94 CIDEr, and the SwiGLU–Mamba-3 module reaches 37.36 at that budget. The unsampled interface thus improves CIDEr by 7.91% over the dense baseline using 112 visual tokens. Stride-two sampling halves the prefix to 56 tokens and yields 37.54 CIDEr. All rows through the unsampled model use the stated training protocol; sampled rows reuse the corresponding 16× weights without further training.

a) Temporal sampling in the full architecture: The last row of Table III examines output sampling with spatial pooling and channel packing present. Sampling changes CIDEr from 35.94 to 35.44 without the temporal block (−1.39%) and from 37.36 to 37.54 with it (+0.48%). Together with the temporal-only sweep, these observed means suggest reduced sensitivity to output sampling. Retained outputs can depend on earlier inputs whose readouts are omitted (Fig. 4), providing a mechanism consistent with the pattern.

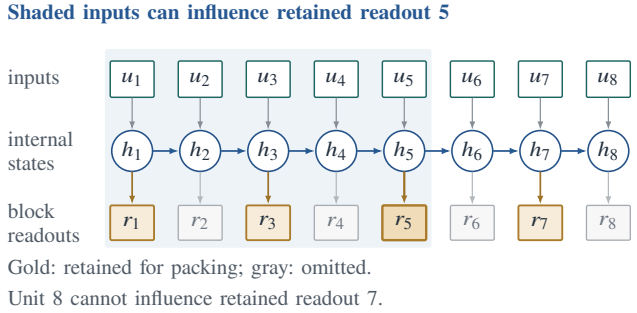


Fig. 4. Aggregate before sampling. This unit-level schematic of the flattened-token scan groups inputs as u_i, internal states as h_i, and final-block readouts as r_i. All inputs update the states; inference keeps readouts from units 1, 3, 5, and 7 for packing. Inputs 1–5 can influence retained readout 5, while trailing unit 8 cannot influence readout 7. Internal states are not passed to the language model.

TABLE IV
CAUSAL AND BIDIRECTIONAL CONFIGURATIONS WITH EQUAL PARAMETER COUNTS. ALL METRICS ARE FIVE-SEED MEANS; CIDER INCLUDES SE.

Configuration	B-4	METEOR	R-L	CIDEr
Bidirectional Mamba	8.32	26.11	28.84	38.35 ± 0.25
Causal Mamba (ours)	8.14	25.93	28.01	37.54 ± 0.32

E. Causal Versus Bidirectional Sequence Modeling

Both configurations in Table IV use four blocks with identical widths, internal settings, and parameter counts, and are trained independently. The causal model scans forward in every block. The bidirectional model alternates forward and backward scans; each backward block reverses the complete flattened sequence and restores output order afterward. Both train without sampling and use stride two at inference. Bidirectional processing reaches 38.35 CIDEr versus 37.54 for causal processing, a 2.16% relative increase. The forward-only interface preserves causal dependence and permits recurrent state reuse; the measured outcome here is complete-clip caption quality.

F. Efficiency Accounting

Table V reports token budgets and theoretical KV storage for the 32-frame configuration, alongside a separate text-only latency diagnostic. Qwen2.5-7B has 28 decoder layers and four KV heads of dimension 128 [4]; BF16 keys and values therefore require $2 \times 28 \times 4 \times 128 \times 2 = 57{,}344$ bytes, or 56 KiB, per token. Compressing the visual prefix from 1,792 to 56 tokens reduces its KV storage from 98.00 to 3.06 MiB. With 30 text/template tokens the total prefixes are 1,822 and 86 tokens, or 99.64 and 4.70 MiB. These are input-prefix figures, not allocated GPU memory: weights, generated tokens, activations, and runtime overhead are excluded.

For the latency diagnostic, we issue text-only requests to Qwen2.5-VL-7B-Instruct on an A800-SXM4-80GB using vLLM, BF16, batch size one, greedy decoding, and prefix caching disabled. Each request uses fresh random token IDs at the matched total-prefix length and generates one token. Over 20 runs after three warmups, mean request latency is 113.5 ms

TABLE V

EFFICIENCY ACCOUNTING FOR THE DENSE AND COMPRESSED PREFIXES. VISUAL AND PREFIX KV ARE THEORETICAL BF16 STORAGE AT 56 KIB/TOKEN; PREFIX COLUMNS INCLUDE 30 TEXT/TEMPLATE TOKENS. LATENCY IS A TEXT-ONLY, ONE-TOKEN DIAGNOSTIC AT THE MATCHED TOTAL-PREFIX LENGTH AND EXCLUDES VIDEO PROCESSING AND THE PROJECTOR.

Configuration	Vis. tokens	Visual KV	Prefix KV	Latency
Dense	1,792	98.00 MiB	99.64 MiB	113.5 ms
Compressed	56	3.06 MiB	4.70 MiB	14.3 ms
Reduction	32×	32×	21.2×	87.4%

for the dense-length prefix and 14.3 ms for the compressed-length prefix, an 87.4% reduction. Timing includes serving overhead and excludes video processing and the projector, measuring the language-model-side response to prefix length.

G. Implications for Collaborative Media Work

A candidate description can serve as a shared draft attached to its source clip. Commentators and editors could verify the action wording against the footage, correct names or details, and reuse the approved text. OnlineSoccer supplies a caption-generation component with an adjustable visual-prefix budget. The temporal module processes the full feature sequence, and the output stride selects how much of the resulting representation enters the language model. This gives the proposed workflow a controllable generation budget while preserving an explicit link between each draft and the clip to be reviewed.

H. Scope and Limitations

The evaluation concerns complete centered clips, including roughly 16 seconds after the annotation time; event-time captioning and streaming latency remain unmeasured. Stride selection used test results, so comparisons describe the selected configurations. Missing per-seed records preclude significance tests, and the temporal-only sweep has no retained uncertainty summaries. Exact seeds, training precision, weight decay, and the generation settings used in the experiments are unavailable for reconstruction. Block ablations evaluate the combined Mamba–SwiGLU module. Direct STORM comparison, cross-domain transfer, and editing effort, factual acceptance, service concurrency, and team productivity in collaborative workflows remain unevaluated.

V. CONCLUSION

OnlineSoccer aggregates temporal evidence before selecting a compact visual prefix. Reduced-width causal processing, spatial pooling, and channel packing yield a 7.91% relative CIDEr improvement over the dense baseline with 16× fewer visual tokens. The test-informed stride-two configuration uses 56 tokens and improves CIDEr by 19.55% over pooling at the same inference budget. The temporal-only sweep further shows a small score range as the prefix shrinks eightfold without retraining. This combination of caption quality, compactness, and adjustable output length makes the interface useful for exploring caption drafting in collaborative sports-media production.

REFERENCES

[1] H. Mkhallati et al., "SoccerNet-Caption: Dense video captioning for soccer broadcasts commentaries," CVPR Workshops, 2023.

[2] J. Rao et al., "MatchTime: Towards automatic soccer game commentary generation," arXiv:2406.18530, 2024.

[3] J. Jiang et al., "STORM: Token-efficient long video understanding for multimodal LLMs," arXiv:2503.04130, 2025.

[4] S. Bai et al., "Qwen2.5-VL technical report," arXiv:2502.13923, 2025.

[5] J.-B. Alayrac et al., "Flamingo: A visual language model for few-shot learning," NeurIPS, 2022.

[6] J. Li, D. Li, S. Savarese, and S. Hoi, "BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models," ICML, 2023.

[7] H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual instruction tuning," NeurIPS, 2023.

[8] J. Cha, W. Kang, J. Mun, and B. Roh, "Honeybee: Locality-enhanced projector for multimodal LLM," CVPR, 2024.

[9] M. S. Ryoo et al., "TokenLearner: What can 8 learned tokens do for images and videos?" arXiv:2106.11297, 2021.

[10] S. Ren et al., "TESTA: Temporal-spatial token aggregation for long-form video-language understanding," arXiv:2310.19060, 2023.

[11] Y. Li, C. Wang, and J. Jia, "LLaMA-VID: An image is worth 2 tokens in large language models," ECCV, 2024.

[12] L. Chen et al., "An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models," ECCV, 2024.

[13] E. Song et al., "MovieChat: From dense token to sparse memory for long video understanding," CVPR, 2024.

[14] X. Li et al., "VideoChat-Flash: Hierarchical compression for long-context video modeling," arXiv:2501.00574, 2024.

[15] M. Cai, J. Yang, J. Gao, and Y. J. Lee, "Matryoshka multimodal models," ICLR, 2025.

[16] X. Shen et al., "LongVU: Spatiotemporal adaptive compression for long video-language understanding," arXiv:2410.17434, 2024.

[17] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," arXiv:2312.00752, 2023.

[18] T. Dao and A. Gu, "Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality," arXiv:2405.21060, 2024.

[19] A. Lahoti et al., "Mamba-3: Improved sequence modeling using state space principles," arXiv:2603.15569, 2026.

[20] L. Zhu et al., "Vision Mamba: Efficient visual representation learning with bidirectional state space model," ICML, 2024.

[21] Y. Liu et al., "VMamba: Visual state space model," NeurIPS, 2024.

[22] K. Li et al., "VideoMamba: State space model for efficient video understanding," arXiv:2403.06977, 2024.

[23] J. Chen et al., "VideoLLM-online: Online video large language model for streaming video," arXiv:2406.11816, 2024.

[24] X. Chen, K. Tao, K. Shao, and H. Wang, "StreamingTOM: Streaming token compression for efficient video understanding," arXiv:2510.18269, 2025.

[25] H. Zhang et al., "HERMES: KV cache as hierarchical memory for efficient streaming video understanding," arXiv:2601.14724, 2026.

[26] G. Xiao et al., "Efficient streaming language models with attention sinks," ICLR, 2024.

[27] Z. Zhang et al., "H2O: Heavy-hitter oracle for efficient generative inference of large language models," NeurIPS, 2023.

[28] Y. Li et al., "SnapKV: LLM knows what you are looking for before generation," NeurIPS, 2024.

[29] S. Giancola et al., "SoccerNet: A scalable dataset for action spotting in soccer videos," CVPR Workshops, 2018.

[30] J. Rao et al., "Towards universal soccer video understanding," CVPR, 2025.

[31] L. You et al., "TimeSoccer: An end-to-end multimodal large language model for soccer commentary generation," arXiv:2504.17365, 2025.

[32] R. Vedantam, C. L. Zitnick, and D. Parikh, "CIDEr: Consensus-based image description evaluation," CVPR, 2015.